

CIHJT	UUHML	FRUGC	ZIBGD	BQPNP	PDNKG	LPLLP	YJYXM
DCXAC	JSJUK	BIOYT	MWQXP	DLIRC	BEXYK	YKIMB	TYIPE
UOLYQ	OKOXH	PIJKY	DRDBC	GEFZG	UACKD	RARCD	HBRYI
DZJYO	YKAIE	LIUYW	DFOHU	IOHZY	SRNDD	KPSSO	JMPQT
MHQHL	OHQQD	SMHNP	HHOQH	GXRJP	XBXP	LLZAA	YCMOG
AWSSZ	YMFNI	ATMON	IXPBY	FOZLE	CYYSJ	XZGPU	CTFQY
HOYHU	OCJGU	QMWQY	OIGOR	BFHIZ	TYFDB	YBRMN	XNLZC

Fig. 2. Depiction of pad used in one-time-pad cryptosystem

tions intercepts. Over a period of years, various secret communications were compromised. This operation took place under the code word 'VERONA.'

If both plaintexts are in a natural language (English or Russian, such as in the above case), even though both are secret, each stands a very high chance of being recovered by cryptanalysis, with possibly a few ambiguities. Of course, the longer message can only be broken for the portion that overlaps the shorter message, plus perhaps a little more by completing a word or phrase.

Working of one-time-pad cryptosystem

The one-time-pad cryptosystem was developed with the following objectives:

1. Unconditionally secure if, no matter how much ciphertext a cryptanalysis has, there is not enough information to recover the plaintext
2. Computationally secure if it cannot be broken with available resources, either current or future
3. Security by obscurity doesn't work

Fig. 1 depicts working of one-time-pad cryptosystem. Each generated key contains a status, to identify own and imported keys and their validation, and the segment information, to store information about the already used bytes of that key. This ensures that each encryption uses another, unused part of the pad. This avoids the most common error in implementing one-time pads (using the same random bytes twice).

Pad generation. Pads can be 100, 1000, 10000, 100000 or more characters long. For pad generation, cryptographically secure pseudo-random number

generation can be used. A typical pad is shown in Fig. 2.

According to Kerchoff's assumption, secrecy must solely reside in the key because the attacker knows the complete details of the cryptographic algorithm. Common consumer items that can be used to transport one-time-pad data are shown in Fig. 3. Note that it is the pad (or the keys) that is transferred.

An important thing to note is that once the particular line and number of the pad are used for any operation (encrypt or decrypt), the particular sequence is blocked and cannot be reused. For this, a block file is used. The block file is shown in Fig. 4. Here



Fig. 3. Common consumer items that can be used to transport one-time-pad data

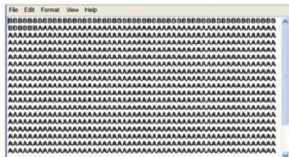


Fig. 4. A typical block file

'B' stands for blocked and 'A' stands for available.

For decryption, a similar sequence (that is, line and column) is required. A facility to acquire the pads or rather import the pads from the other system can also be inculcated. Properly used one-time pads are secure against adversaries with infinite computational power. Normally, modulo 28 and modulo 26 operations are used for encryption/decryption process, but many other techniques are also available.

The sender and the receiver agree upon a one-time pad for message exchange. Therefore the first group of the one-time pad is used as the key indicator at the beginning of the message. A generally accepted security measure is to avoid the usage of the first group of the pad in the encryption process. One-time-pad serial number is not sent along with the message (as a security measure) as it can reveal the number of messages that were sent, and also their order.

An example of one-time-pad encryption is shown in Fig. 5. Here we have taken 'HELLO' message to be encrypted with the key as 'XMCKL'. Using modulo 26 ($A=0, B=1, \dots, Z=25$) we try to encrypt the message. This can be further Base64 encoded for the final output. Note that if a number is larger than '25', in modular arithmetic fashion, '26' would be subtracted from the number to make it less than '26.'

Similarly, one-time pad decryption is shown in Fig. 6. The cipher message, i.e., 'EQNVZ,' is decrypted using the same key 'XMCKL' and modulo 26. If a number is negative, '26' is added to make the number non-negative.

Using the modulo 26 ($A=0, B=1, \dots, Z=25$)